LAB 5 – SSIS

CONTENT

- Introduction to SSIS, SSIS Packages
- Control Flow in SSIS
- Data Flow in SSIS

ADDITIONAL RESOURCES

- SSIS Introduction - basics of SSIS packages:
  - Introduction to SSIS: https://docs.microsoft.com/en-us/sql/integration-services/sql-server-integration-services
  - How to create ETL package: https://docs.microsoft.com/en-us/sql/integration-services/ssis-how-to-create-an-etl-package
  - SSIS packages: https://docs.microsoft.com/en-au/sql/integration-services/integration-services-ssis-packages
- SSIS Introduction - Creating SSIS Package:
  - SSIS designer interface: https://docs.microsoft.com/en-au/sql/integration-services/ssis-designer
  - SSIS create ETL package - lessons: https://docs.microsoft.com/en-au/sql/integration-services/create-packages-in-sql-server-data-tools
  - SSIS add existing package: https://docs.microsoft.com/en-au/sql/integration-services/add-copy-of-existing-package
  - Control Flow details: https://docs.microsoft.com/en-au/sql/integration-services/control-flow/control-flow
- SSIS on-line tutorials:
  - http://www.codeproject.com/Articles/155829/SQL-Server-Integration-Services-SSIS-Part-Basics
  - https://www.mssqltips.com/sqlservertutorial/9053/sql-server-integration-services-ssis-2016-tutorial/
  - https://www.tutorialgateway.org/ssis/
- SSIS Data Flow - Basic Transformations
  - https://docs.microsoft.com/en-us/sql/integration-services/data-flow/transformations/integration-services-transformations?view=sql-server-2017
    - Focus on Aggregate, Conditional Split, Derived Column, Lookup, Merge, Merge Join, Multicast, Pivot, Row Count, Sort, Union all
- Variables
  - https://docs.microsoft.com/en-us/sql/integration-services/integration-services-ssis-variables?view=sql-server-2017
- Event handlers
  - https://docs.microsoft.com/en-us/sql/integration-services/integration-services-ssis-event-handlers

TUTORIAL 0 – INTRODUCTION TO SSIS

Please look at the presentations on SSIS. The presentation files are available on ePortal.

TUTORIAL 1 – DATA MOVEMENT AND FIRST SSIS PACKAGE

Please follow this basic introduction to SQL Server Integration Services (SSIS) using SQL Server Data Tools (SSDT):

1. Using SQL Server 20XX Import and Export Wizard try loading tables Production.Product, Production.ProductSubCategory, Production.ProductCategory into your database (please use a new schema name [Source] or prefix/suffix to table names).

   a. During the import/export process creation remember to select "Save SSIS Package" option, choose file storage (not in SQL Server) and save the file on the desktop or some other known location.
2. Further please run Visual Studio and create a new project – Integration Services Project.
   a. For additional details, please take a look at: https://www.tutorialspoint.com/ms_sql_server/ms_sql_server_integration_services.htm
3. In the Solution Explorer pane (right side of the window) under the SSIS Packages right click and select Add Existing Package. Attach the newly created .dtsx file (see step 1a). You should now see the tasks that are involved in the import/export process you've just defined.
   a. Try to analyse the attached package, go into details of each task (double click on the *blocks / right click and select Edit*) and all precedence relations (double click on the *arrows*) and try to answer these questions:
   b. What tasks are involved? What do they do? Is the execution (do not run it yet) of these tasks somehow ordered? What type of ordering is utilised?
   c. Rename this package to [ImportProductInformation].
4. Remove, manually, the newly created tables from your database (the ones with [Source] schema or prefix/suffix). Further, try running the task by using the Start button. The view changes into the run mode – notice there is a new tab available – Progress.
   a. What information is displayed there? Can you identify the text output of the results of running the package? HINT – look at bottom right part the window.
   b. Close the run mode by pressing red square button (Stop), and you'll be back in the design mode.

TUTORIAL 2 – SSIS BASICS – CONTROL FLOW

Control Flow – establishing the precedence relation and run basic control statements. Please read the general information regarding the Control Flow aspect of the SSIS package:

- https://docs.microsoft.com/en-us/sql/integration-services/control-flow/control-flow
- https://docs.microsoft.com/en-us/sql/integration-services/control-flow/add-or-delete-a-task-or-a-container-in-a-control-flow

1. Within the project from the previous task create a new package (Solution Explorer pane), name it [Start]. Within this package drag&drop the Execute Package Task from the SSIS Toolbox.
   a. Try to configure this task to run the [ImportProductInformation] SSIS package from the same project (solution). Open details and select, in the Package tab (left), the reference type to "Project Reference" and package name (PackageNameFromProjectReference) to "ImportProductInformation".
   b. Test your [Start] package, by running it.
2. Now create a new package in the same project and name it [SQLIntro]. Within this package drag&drop the Execute SQL Task.
   a. Set this task to create a basic schema in your database (use name [SSISTest] for the schema) on the database server (same as the one you accessed using MS SQL Management Studio).
      i. As such, you need to set the connection type to OLE DB and establish connection with your database (Connection option allows for creation of a new connection or for using an already established one). Further select SQL source type as Direct Input (you will provide inline SQL statement) and in the SQL Statement option provide a T-SQL statement, which creates a new schema. For additional details look at:
         1. https://docs.microsoft.com/en-us/sql/integration-services/control-flow/execute-sql-task
         2. https://docs.microsoft.com/en-us/sql/t-sql/statements/create-schema-transact-sql
   b. Further, add another Execute SQL Tasks. This task should create a needed table (name it [ProductTest], use the newly created schema), use CREATE TABLE T-SQL statement.
      i. The table should consist of 5 columns – ProductID, ProductName, ProductSubcategoryName, ProductCategoryName, ProductColor.
   c. We want the schema to be created before creating the table, as such please add an arrow between the former and the latter Execute SQL Task.

   d. Run the current package – note that the Start option might be running the default package and not the current one, as such you might want to right-click on the empty design pane and select Execute option.
      i. Verify that the table was created in the database – you can either use Management Studio or Server Explorer (within Visual Studio) to connect to the database server.
      ii. Further, run this package multiple times – you should get an error stating that the structures you are trying to create already exist. If so, please do not fix this error, just disable the schema and table creation tasks (right-click on them and select Disable).
3. Further, add another Execute SQL Task to add data to the [ProductTest] table. We want to add some basic rows, i.e., move and join the imported data into the new table using INSERT INTO statement.
   a. Prepare required T-SQL Statement and set a proper precedence relation with the already defined tasks.
   b. Run the package
      i. Run it once - did everything run correctly? Check data.
      ii. Try running the package again (as the creation tasks are disabled you shouldn't get any additional errors) – what do you get as a result? Look at the data within the tables – try running the package multiple times.
4. Add another Execute SQL Task to establish the ProductID as the primary key of the [ProductTest] table. Use ALTER TABLE statement https://docs.microsoft.com/en-us/sql/t-sql/statements/alter-table-transact-sql.
   a. Prepare required T-SQL Statement.
   b. Should there be any precedence relation between the two tasks? If so create one.
5. Now please enable the schema and table creation tasks (right-click on them and select Enable). Now let us modify the package, by adding additional Execute SQL Task(s) at the beginning of the control flow, to drop the schema and the table if they already exists. HINT: You can use IF EXISTS option (https://www.tech-recipes.com/rx/68871/drop-if-exists/).
   a. Set proper precedence relations between the tasks within this control flow.
6. Finally, please enable logging to an external file in the [SQLIntro] package.
   a. You do this by using SSIS --> Logging menu option. Please follow the tutorial available at https://www.tutorialgateway.org/ssis-logging/ (start at Configure Logging in SSIS step). Please remember to store log entries only when an error occurs, immediately before the executable runs, and immediately after the executable has finished running.
   b. For more information look at SSIS logging capabilities - https://docs.microsoft.com/en-us/sql/integration-services/performance/integration-services-ssis-logging
2. Run the package - Can you identify how long it took for the individual tasks to run?

TUTORIAL 3 – DATA FLOW- TUTORIAL

In the previous tutorial we focused on using predefined SQL queries to transform and move the data around and used SSIS Control Flow to orchestrate these queries. Here we'll focus on enriching the original process by introducing an additional data processing steps, but now all transformation steps will be designed within Data Flow part of SSIS, without any need of SQL or programming needed.

1. Create a new SSIS package. Within this package add a new Data Flow task (from SSIS Toolbox) – name it [SalesTerritoryFlow]. Double click on the newly created task (or use the Data Flow tab and select SalesTerritoryFlow from the available combo box). Please note that within the Data Flow editor view, there is a completely new SSIS Toolbox – you currently have access to Sources (connect to external sources and provide access to the data; which output data), Transformations (connect to outputs of other tasks, and generate output for other tasks) and Destinations (consume data from other outputs; generate no regular outputs). Remember that within Data Flow, each connection between tasks/processing steps represents data flowing from one task/step to another; blue represents the regular data flow, whereas red represents erroneous data flow.

2. Within Data Flow editor:

   a. Add a new Data Source (OLE DB source). Next, in Connection Manager tab, specify connection details to connect to AdventureWorks database and SalesTerritory table. Next. In the Columns tab you can specify, which columns will be accessible in the output. Please note that you can also provide connection details using SQL queries (we are not using this option now).

b. Add a new Transformation task (Sort). Before specifying its details remember to connect blue output arrow from the Data Source task with the Sort task (just drag&drop it). We want the system to know the metadata of the anticipated data flow. Double click on the Sort task (or right-click and choose Edit). In settings menu, just specify to order data based on CountryRegionCode (tick the proper box). Please note that Pass Through column specifies which attributes are available in the output – select all but SalesYTD, SalesLastYear, CostYTD and CostLastYear. Rename the task to SalesTerritorySort.

c. Add a new Data Source (OLE DB source). Next, in Connection Manager tab, specify connection details to connect to AdventureWorks database and choose CountryRegion table. Select all available columns.

d. Add a new Sort task, connect it to the data source and sort the CountryRegion data based on the CountryRegionCode. Rename it to CountryRegionSort.

e. Add a new Merge Join task (which requires both inputs to be sorted based on the join key). Connect it to both Sorts: SalesTerritorySort (left) and CountryRegionSort (right). In the Merge Join task details, set Join Type to "Left outer join" and select CountryRegionCode as the join key (in both tables, if not set automatically). Next select all columns, beside CountryRegionCode, as the output. Rename the Name attribute to CountryName.

i. For details look at: https://www.tutorialgateway.org/merge-join-transformation-in-ssis/

f. Add a new Data Destination (OLE DB destination). Next, connect blue output arrow from the Merge Join task with the Data Destination task (just drag&drop it). Double click on the Data Destination tasks and specify connection manager (to your database), as data access mode select table or view. Select New, next to the name of the table or the view – and create a new table, as suggested by the SSIS tool (just change the table name to SalesTerritoryInformation). Next check mapping (you need to do this, as the mapping is not created without entering the mapping tab on left).

i. Note that table creation using the New option is only executed once during at the design time. If later removed, the table will not be created by the package – if needed you should include proper SQL statements in the Control Flow part of the package (and here you need them).

ii. For details look at: https://www.tutorialgateway.org/ssis-ole-db-destination/

3. Right click on the blue arrow between Merge Join and Destination, and Enable Data Viewer (alternatively, use Edit and check the Data Viewer option. Run the package (using the green play button) or the Data Flow task (right click on the task and select Execute)– note that a new pop-up should appear – is the data sorted correctly?

a. Please note that Data Viewer a strictly debugging tool, as such during execution, to continue data processing, you might need to press the green play arrow on a pop-up screen.

4. Run the package multiple time. Check for errors and look at the output table – is the data correct?

TUTORIAL 4 – VARIABLES BASICS - TUTORIAL

Variables in SSIS can also be used to perform basic sanity checks – here we'll prepare an automated process for a simple test of the number or rows in the source tables (extracted) against prepared destinations (inserted).

1. First, let us count the number of extracted rows.
   a. Introduce a new variable ExtractRowCount to store information about number of records in the source data file before staging the data into staging tables.
      i. Use SSIS -> Variables menu available under Extensions > SSIS > Variables. A new window/pane should appear with all user variables.
      ii. Add new variable, set name to ExtractRowCount, set data type to Int32, and default value to 0 – do not specify any expression.

   b. Within the SalesTerritory Data Flow task add a Row Count Task and assign it's result to ExtractRowCount variable.
      i. Note that Row Count Task acts as a simple meter – it takes the input flow, counts the number of rows and stores this number in a variable, and outputs the input flow.
2. Now, let us count the number of inserted rows.
   a. Create a new variable InsertRowCount to store information about the number of records which was added to the table.
   b. We will implement the solution using a simple Execute SQL Statement (you could also do it using Row Count Task).
      i. Create a new Execute SQL Statement task.
         1. As the SQL query prepare a query to count the number of rows in the destination SalesTerritory table. Make sure that only the number of rows is return (single value and a single attribute).
         2. As the ResultSet select Single Row Result Set
         3. In the Result Set tab (left) add new result-variable mapping pair: set result name to 0 (first attribute) and variable to User::InsertRowCount.
3. Create a new empty „Test" data flow task within your package.
   a. Make sure that the Test task (dummy task in our package) will run only after both variables Extract- and InsertRowCount are set.
   b. Use a proper evaluation operation type for the arrow/connection (double click on the arrow/path and change Evaluation operation to Expression).
   c. As an expression compare both variables. Use the == operator, and drag&drop variables from the list of available variables.
4. Run the task. What happens? Is everything alright? Can you preview the values of variables during execution? Hint – use Breakpoints.

(*) TUTORIAL – EVENT HANDLERS BASICS - TUTORIAL

Event handlers are a special type of SSIS control flow. An event handler is designed to execute when a specific event occurs. Allow to define additional tasks that can run at specific situations in execution of SSIS packages – like PreExecution, PostExecution, OnError, etc. Event handlers are useful tools but be careful that you don't overuse them. Because the event handler logic can be set up individually for every task and container (as well as for the package itself), the overuse of event handlers creates a labyrinth of logic that is very difficult to maintain and debug.

SSIS event handlers - https://www.red-gate.com/simple-talk/sql/ssis/ssis-event-handlers-basics/

During ETL, we should store process metadata (ideally about every row). This may include: name of the job that inserted the row, identifier of the job execution, date and time when it was executed, name of the source system or source file, user who executed the job, number of processed rows. This information is invaluable both for regular monitoring as well as debugging when something goes wrong. Think about a very simple example - when somebody loads a wrong source file by mistake, how can you quickly identify rows that should be deleted with no audit dimension?

Here we'll introduce a modification to an already existing package to generate some basic process metadata. To do so we'll use event handlers.

1. Create a table in your database:
   a.
```
CREATE TABLE dbo.BasicAudit (
 [AuditKey]  int IDENTITY NOT NULL
, [PkgName]  nvarchar(50) DEFAULT 'Unknown' NOT NULL
, [PkgGUID]  uniqueidentifier   NULL
, [ExecStartDT]  datetime  DEFAULT getdate() NOT NULL
, [ExecStopDT]  datetime   NULL
, [TableName]  nvarchar(50) DEFAULT 'Unknown' NOT NULL
```

```
, [ExtractRowCnt]  bigint   NULL
, [InsertRowCnt]  bigint   NULL
, [SuccessfulProcessingInd]  nchar(1) DEFAULT 'N' NOT NULL
, CONSTRAINT [PK_dbo.BasicAudit] PRIMARY KEYCLUSTERED ( [AuditKey] ) ) ON [PRIMARY];
```

2. Use the SSIS package from the previous lab assignment.
    a. If, for some strange reason, you do not have it, please create a basic SSIS package that extracts data from Production.Product and Production.ProductSubcategory, merges both (use left join) and outputs to a new table.
3. Within your project add OnPreExecute handler to the first data flow task in the package. The handler should create an entry within the BasicAudit table and fill-in basic information about the package. To do so:
    a. Change your tab to Event Handlers (within SSIS Designer). From Executable combo box, open the package level and executables, and select the first data flow task. Within the Event handler combo box select OnPreExecute. Click on the "Click here to create an 'OnPreExecute' event handler for …" text to create a new design pane.
        i. Note that you can add event handlers to the package (even on the package level).
        ii. Note there are also other events available - https://docs.microsoft.com/en-us/sql/integration-services/integration-services-ssis-event-handlers?view=sql-server-ver15#run-time-events
    b. Next, create an Execute SQL Task task and edit it.
        i. Use ADO.NET connection type and set connection to your database (which contains the BasicAudit table). This time we would like to use some of the system variables available in SSIS to store basic process metadata within the BasicAudit table. In order, to do so please open Parameter Mapping tab. Select needed parameters:
            1. Use system variables (https://docs.microsoft.com/en-us/sql/integration-services/system-variables) – i.e., PackageID, PackageName, StartTime
            2. Set Direction as Input and set meaningful names.
            3. Note – we are using ADO.NET connection type to use simplified references to individual parameters "@ParamName", otherwise we would need to use the order of definition of parameters "0","1", etc.
        ii. Next, write a simple INSERT INTO statement to add row to the BasicAudit table – use "@ParamName" to reference input parameters.
            1. Using parameters from PackageID, PackageName, StartTime system variables and set the values of first 4 attributes of BasicAudit table – the rest can be set after the execution of the package.
4. Run the package and see if any new rows appear in the BasicAudit table.

Next we'll extract the value of the current [AuditKey] (auto numerated field) and store it in a local variable:

1. Introduce a new Variable within the project - AuditKeyVar to store information about current audit key. This key will further allow us to update the metadata, after package execution or on error.
    a. Use SSIS -> Variables menu available under Extensions > SSIS > Variables. A new window/pane should appear with all user variables.
    b. Add new variable, set name to AuditKeyVar, set data type to Int32, and default value to 0 – do not specify any expression.
    c. Add new variable, set name to AuditSuccessStatus, set data type to Char, and default value to "T" – do not specify any expression.
2. Return to the event handler from the previous task and add new Execute SQL Task task. This new task should run after the successful execution of the former one.
    a. Connect to your database – database which contains BasicAudit table
    b. In the general tab change the Result Set type to single row – as we want to retrieve data from a single row of result set.

c. Prepare adequate SQL query to get the AuditKey value from the last row in BasicAudit table – make sure to assign a name to each column in the resultant relation of SQL Query. As ResultSet type is Single Row – the query should provide only a single row.
d. In the Result Set tab assign the resultant column to the variable AuditKeyVar. Please remember to use the resultant column name for assignment.

Finally, we can add missing data to BasicAudit table using the current [AuditKey] from the previous steps.

1. Add a new OnTaskFailed handler. This handler should set the value of the AuditSuccessStatus to "F".
    a. Hint – use Expression Task, AuditKeyVar, and CODEPOINT() (T-SQL function)
2. Add a new OnPostExecute handler. This handler should insert the missing data to the BasicAudit table.
    a. Hint – use GETDATE() to get stop time, AuditKeyVar, and NCHAR() (T-SQL function)
3. Run the task.

TASK 1 – SSIS BASICS – ETL FLOW MANAGEMENT

Using SQL statements in SSIS – Control Flow management.

The idea is to implement an automated data pipeline. The pipeline should result in the basic star schema from the previous laboratory assignment (Sales, Time, and Product). In your implementation, please create a snapshot of the source AdventureWorks database into a staging database (created for this task), do data transformation using SQL statements (remember to clean and conform data), clean the staging environment (if necessary), load data into final data storage solution (here, a database with user access, storing data in a user-ready star schema).

Note: *In this task do not use external product review data.*

Your task is to:

1. Create a staging database and create a new Integration Services Project (Visual Studio).
2. Create a SSIS Package to snapshot the source AdventureWorks database tables required for star schema creation within staging database. You can use your own SQL or SSIS package created by the Import/Export wizard tool.
3. Create a SSIS Package to do data transformation within staging database. You should be using SQL statements and processes defined in the previous laboratory assignment (set of SQL queries for creation of a simple star schema). Define a control flow with several Execute SQL tasks and proper precedence relations between them, please do not use a single Execute SQL task (with all queries) - try organising your data processing tasks into logical groups (further utilise the ability to run some of the processing tasks in parallel). Finally justify your approach (implementation in SSIS environment).
    a. Please do not introduce additional Data Flow tasks and stick to pure SQL based transformations and operations on the data – you already have all the needed queries; we are just trying to organize and automate the entire process. As such, focus on using Control Flow part of SSIS to manage the flow of control of your SQL statements.
    b. Please explain your design decisions.
4. Create a SSIS Package to load the data into destination, i.e., dimension and fact tables in a dedicated database/schema.
5. Create a master SSIS Package to run other three packages (Extraction, Transform and Load). Use Execute Package task to execute other packages from the same SSIS project. This package is a form of a master package to run all other tasks, i.e., extraction, transformation, and loading.
    c. Try running the package in three situations: destination database is empty, destination database contains empty tables (only structure), destination database contains tables already containing data (structure + data). Comment the results. If needed introduce proper changes in the previous packages to handle all three situations.

You can create a different structure of the project, without multiple individual SSIS packages for each stage.
Just explain and argument your decision and approach in the report. As a result, please submit report and .dtsx files (SSIS Package files).

TASK 2 – DIMENSIONAL MODEL USING DATA FLOW

We have already developed the process of preparing AdventureWorks data – creating a simple star schema comprised of a Product and a Time dimension, and product's sales data (as in the previous laboratory assignment). In the previous laboratory assignment, we've focused on using predefined SQL queries to move the

data around and SSIS Control Flow to orchestrate these queries. Here we want to enrich the model to include SalesTerritory dimension (representing sales territory assigned to a particular order-item) and SalesPerson dimension (representing sales representative assigned to a particular order-item). We want to create these dimensions within Data Flow part of SSIS, without any SQL.

Now let us extend the capabilities of the SSIS package to introduce some data flows. To do so we'll use the Data Flow Task – you can simply drag&drop it into the Control Flow pane. Notice that when you try to edit this task (after double clicking it) the project pane switches to Data Flow tab and completely new tasks are available. This is the Data Flow view of the project. Here there are 3 types of tasks – source, destination, and transformation. Source tasks are utilised to start a data flow and extract data from a specified data source, destination tasks are the final nodes where the data flow ends, and the flowing data is exported to the specified destination. Whereas the transformation task, take the incoming data flow, perform some operation on it, and output a modified data flow (sometimes multiple). Bear in mind that in this view (Data Flow) "the arrow" (path) represents the actual flow of data (different from Control Flow – where it represents the precedence relation). There are in principle two types of flows of data: "correct one" (blue), as the regular data flowing over the path, and the "erroneous one" (red), as these are the data elements that generated some kind of exception/error during the processing task within the block ("source", "transformation" or "destination" operation). Please note, that by default all errors are causing a block to fail the component, i.e., generate an error and throw an error event (which causes the entire process to fail). You can change the default behaviour in the Error Output pane of the block settings/properties/editor window (the window you use to configure the task).

Let us now create two additional dimensions:

1. *Create a Data Flow task to prepare a SalesPerson dimension, containing sales person details (please add sales person full name). Within this process you should connect to the required view (to simplify the task use vSalesPerson), clean and standardize the data (remember to introduce calculated attribute - fullname, use Derived Column task), and additionally enrich the data by introducing a new row to the dimension representing missing/unknown sales territory.*
2. *Create a Data Flow task to prepare a SalesTerritory dimension containing sales territory details (please add Country name attribute. Within this process you should connect to the required tables, merge these tables (use Merge Join task or Lookup), clean and standardize the data (use Derived Column), and enrich the data by introducing a new row to the dimension representing missing/unknown sales territory.*
3. *Modify fact creation SQL to include SalesTerritory identifier and SalesPerson identifier – to be able to create a star schema.*

Please, prepare a short report with implementation details and upload as the solution.

(*) TASK 3 – DIMENSIONAL MODEL

In such a setting, let us assume that we want to add additional information about the facts. We have managed to get access to data (from an external system) about product's rating (rating provided by users). The data is available as a series of CSV files (available on EPortal). Please look at the available data and consider how this information could be used.

*Let us now try to incorporate the collected information about products' ratings (*We *have managed to get access to data (from an external system) about product's rating (rating provided by users* on external website*)* – a CSV *file (EPortal). Assume we want to extend the original data model (from previous task) to include some information from the rating dataset.*

In general, you need to extract and transform the data and further load them into extended dimension tables. To simplify the task let us focus solely on extracting aggregated product review data and incorporating it in the already available Product dimension, i.e., we want to add average rating, min rating, max rating and popularity score as novel attributes describing each product. Additionally, as not all products were reviewed, we will add an auxiliary attribute hasReview to reflect whether a product was reviewed at least once ("has reviews") or not ("no reviews").

As a result, please use .dtsx file (SSIS Package) as the solution. You can create a single file for the entire Task 1 (parts A, B and C) or three separate files for each part.

a. Create a new SSIS project and an empty package.

b. Add a new Data Flow task - use "Extract" as the name – to extract the data from external CSV into the staging database.

a. From Other Sources (in the SSIS Toolbox) select the Flat File Source:
   i. Define connection to rating data file.
      1. Make sure to specify proper column delimiters; Use proper code page and use Suggest Types option (based on 200 rows) to infer column data types
   ii. Select all columns
   iii. In ErrorOutput specify to redirect rows on all truncation problems – select all cells in the truncation column and change action to "Redirect Row"

a. Further introduce an additional task Sort and join the Source with the Sort task using the blue arrow. Configure the Sort task to order the data by product id – make sure to output all columns.
   i. Right click on the blue arrow between Source and Sort and select Enable Data Viewer. Now run the task and interpret the results. Is everything correct? Are the previewed results sorted? If not, what is wrong here?

c. Add a Flat File Destination to store the results of the error output of the Flat File Source task.

d. Move the sorted dataflow to a new table:
   iv. Add OLE DB Destination to the package and join the current data flow with the Destination task using the blue arrow. Define the destination (during the process of defining OLE DB Destination create a table ExtractRating). Use your own database. Make sure to specify the correct mapping
   i. Add additional data viewers on all arrows and run the task. Make sure to take a look at the created table in the database. Try running the task multiple times and look at the number of rows in the destination table: What happens here? Further, try running the task in a situation when the resultant table ExtractRating doesn't exist. What happens here? Afterwards, recreate the table.
      1. Please look at http://thesqlgirl.com/2016/09/17/ssis-package-validation/ - to learn about ability to delay validation of tasks and components.

e. Add Execute SQL Task (in Control Flow) that will run before the above data flow and allows to truncate the ExtractRating table:
   i. Use TRUNCATE - https://docs.microsoft.com/en-us/sql/t-sql/statements/truncate-table-transact-sql?view=sql-server-2017

f. (*) Add a Script Component
   a. Join the Script Component with the Source using the red arrow
   b. Edit the Script Component
      i. In Input Columns select ErrorCode and ErrorColumn
      ii. In Input and Outputs add new Output Columns – use columns
         1. Define ErrorCol as UnicodeString of length 150
         2. Define ErrorDesc as UnicodeString of length 500
         3. Change *SynchronousInput* to *None*
      iii. In Script click Edit Script
         1. In public *override void Input0_ProcessInputRow(Input0Buffer Row)* method insert the following lines of C# code to extend the basic information with error column name and error description (instead of generic code numbers)

```
Output0Buffer.AddRow();
Output0Buffer.ErrorDesc = this.ComponentMetaData.GetErrorDescription(Row.ErrorCode);
IDTSComponentMetaData130 componentMetaData = this.ComponentMetaData as IDTSComponentMetaData130;
Output0Buffer.ErrorCol = componentMetaData.GetIdentificationStringByID(Row.ErrorColumn);:
```

         2. Save and build the file, then close
      iv. Further introduce an additional task Sort.
         1. Join the Script Component with the Sort task using the blue arrow.
         2. Configure the sort to order the output by ErrorCol – make sure to output all columns.

g. Now run the task and interpret the results.
   i. Is everything correct?

ii. Make sure that you run simple sanity checks and try to validate the completeness of your extraction – like counting the number of rows and comparing the source and destination results.
    1. In a simplified solution try to manually compare the results
    2. In a better solution use variables (either by assigning the SQL query results or by using Row Count task) to store the number of records in the source and the destination, and further use expressions to change the behaviour of the process depending on the result of comparison of the variables (use the expressions you can define over arrows in the Control Flow).

Now focus on preparing the data (doing all of the cleaning and preparing aggregates).

1. Create a new Data Flow task – use "Staging" as name.
    a. Create a new DataFlow task.
    b. Use rating table ExtractRating (from the previous task).
    c. Define a data flow (within a single data flow task). The flow should result in transformations needed to prepare the aggregated review data (Product id, AvgRating, MinRating, MaxRating, RatingCount) – use StageRating table to store results:
        i. Sort data based on Product id
        ii. Clean the data – make sure that you move (to the staging part) only products, which are available in our inventory (it turns out that the website covered several more products)
            1. Hint - use Lookup task to extract only reviews about the available products
                a. Use ProductDim from SQL Server database (previous assignment) as the lookup table.
                b. Eliminate all non-matching rows (you could use a flow to move them into an external file - Use Flat File Destination)
                c. Matched rows should be processed further
        iii. Clean the data – make sure that the reviews are in an appropriate range (0-10) before aggregating the data:
            1. Hint – you can use Conditional Split and/or Derived Column
        iv. Aggregate the data.
            1. Hint – you can use Aggregate Task
        v. Send data to a table "StageRating" (use your own database, or other database you have write permission to). Make sure to specify the correct structure and mapping!
            1. Hint – add proper Control Flow tasks to create the structure.
            2. Hint – during the process of defining details of flow's destination (OLE DB Destination) SSIS you can access needed SQL statements to create the table.
2. Now run the task and check the results.
    a. Is everything correct?

The staging table should contain all the required information extracted from review data.

Having the clean data at hand, we can load them into the dimension table. For the sake of simplicity, we will create a new dimension table called ProductExtendedDIM, which will cover all information from the ProductDIM extended with aggregated review data (prepared in the previous task).

1. Create a new Data Flow task – use "Loading" as name.
    a. Use rating table StageRating and ProductDIM (from previous tasks) as source data.
    b. On your own, please define a data flow that results in a dataset containing all product data merged with the reviews.
        i. Please remember to add the aforementioned attribute hasReviews. Introduce proper business logic to fill it's values.
    c. Store the data in the ProductExtendedDIM table
        i. Make sure to prepare proper structures in the Control Flow.
2. Update the Fact table to reference the extended dim table..
3. Now run the task and check the results.
    a. Is everything correct?

Please, save and upload the resultant .dtsx file (SSIS Package) as the solution.

SIDE NOTE:

Please remember that you could also use the available review data to create a simple star schema using the available rating data. As the focus is still on the analysis of sales (star schema from the previous lab assignment), you would need to make sure that some of the common dimensions are conformed (creating a basic fact constellation schema). In short, you would want to make sure that Product and Time are conformed.

In more details, you could extract and transform the rating data into a very simple multidimensional data model. The default model should be comprised of a single fact (rating) table and three-dimension (product, time and location) tables. Please remember that both, product and time dimension, should be conformed (with corresponding dimensions from the PPA star schema).